UFC Adversarial Judge

Project Documentation: Phase 1 & 2 — Data Pipeline & AI Integration

1. DATA ACQUISITION (THE SCRAPER)

Goal: Extract reliable fighter statistics from UFCStats.com while maintaining a low profile.

`Python` `BeautifulSoup4` `Requests`

Alphabetical Crawl: Systematically iterates through A-Z sub-indices to ensure 100% roster coverage.

Anti-Bot Protection: Implemented `time.sleep()` with randomized intervals (1.0 - 2.5s) to mimic human browsing behavior.

Regex Parsing: Used `re` (Regular Expressions) to extract raw integers from complex HTML strings.

2. DATA ENGINEERING & CLEANING

Goal: Transform messy web data into a "Model-Ready" dataset.

`Pandas` `NumPy`

Metric Standardization: Converted Imperial units (Feet/Inches) to Metric (CM).

> *Metric units provide a continuous linear scale that improves the accuracy of distance-based ML algorithms.*

The "Zero-Data" Filter: Automatically dropped profiles with 0-0 records to eliminate noise from non-competitors.

Median Imputation: Replaced missing values with the dataset median rather than the mean.

> *Medians are more robust against outliers and skewed athletic statistics.*

Feature Engineering: Created `win_ratio` (Wins / Total Fights) to normalize success across different career lengths.

3. MACHINE LEARNING ARCHITECTURE

Goal: Identify correlations between physical attributes and competitive success.

`Scikit-Learn` `Random Forest`

Model Choice: Random Forest Regressor. Handles non-linear relationships effectively (e.g., Reach-to-Height ratios).

Features (X): Height, Reach, Strike Accuracy, Takedown Average, Win Ratio.

Target (y): Total Career Wins.

Interpretability: Utilized `Feature Importance` to rank which variables the AI values most when "Judging" a fighter's potential.

4. DEVOPS & WORKFLOW

`Git` `Branching`

Branch Management: `kaggle-data-integration` for the pipeline; `model-training-logic` for AI experimentation.

Project Structure: Adheres to a modular `/scripts`, `/data/raw`, and `/data/processed` hierarchy.